Electronics and Arduino Fundamentals

This document is a set of notes covering fundamental concepts in basic circuitry and an introduction to the Arduino system.

1. Basic Circuitry Knowledge

- ### voltage, current, and resistance & ohm's Law

  Voltage (V): the electric potential difference between two points in a circuit - the push or the force that causes current to flow. Measured in Volts.

  Current (I): the rate of flow of electrons through a conductor. Measured in Amperes (Amps).

  Resistance (R): a material's opposition to the flow of current. Measured in Ohms.

  Ohm's Law: $V = I * R$

  Other variables: $I = V / R$; $R = V / I$

- ### Power, including DC vs AC, and safe levels

  Power (P): the rate at which electrical energy is transferred by an electric circuit. A measure of how much work a circuit can do. Measured in Watts (W).

  $P = V * I$

  Direct Current (DC): the electric charge flows in one direction only.

  Alternative Current (AC): the direction of the electric charge **periodically** reverses. More efficient for long-distance power transmission, but must be converted to DC for most electronic devices.

  Safe levels: Any voltage above 50V AC or 120V DC is generally considered hazardous. Arduino and pods - typically use very safe low voltages (5V or 12V DC). Current over 10-30 mA passing through the human body can be harmful or fatal.

- ### Series vs Parallel circuits (voltage and current dividers)

  Series Circuit: components are connected one after the other in a single path.

  The current (I) is the same at every point in the circuit.

  The voltage (V) is divided among the components - total voltage is the sum of the individual voltages across each component.

Resistance: R_total = R1 + R2 + … (the sum of the individual resistances.)

Voltage Divider: two series resistors used to produce an output voltage that is a fraction of its input voltage.

Parallel Circuit: components are connected across the same points, creating separate paths (branches) for the current.

The current is divided between the branches.

The voltage is the same across all components.

The total resistance is less than the smallest individual resistance.

1/R_total = 1/R1 + 1/R2 + ….

Current Divider: The current in a parallel branch is a fraction of the total current.

- ### How buttons and switches work

Electromechanical components used to control the flow of the current in a circuit by either opening or closing a circuit. Connect or disconnect two parts of a circuit.

Momentary Switch - Button: only maintains its contact state while actuated, open or close.

Toggle Switch - Switch: Maintains its state after being actuated, e.g. ON/OFF switch.

Arduino: when a button is used with an Arduino, it is typically connected to a digital input pin. Use a pull-up or pull-down resistor to prevent the input pin from 'floating', which can lead to erratic and unreliable readings. The Arduino Uno offers **internal pull-up resistors**, which can be activated in the code - using pinMode(pin, INPUT_PULLUP)

2. Arduino System (General Information)

- ### What it is (overview)

Arduino https://docs.arduino.cc/learn/ is an open-source electronics platform (hardware + software). A small, simple computer that can be programmed to interact with the real world.

It is a **microcontroller board** used to read inputs and turn them into outputs. Inputs include things like from sensors, buttons, or simple commands. Outputs include things like controlling motors, turning on LEDs, or sending data. Ideal for prototyping and educational projects.

- ## Basic parts of the Arduino board (Arduino Uno R3)

  Microcontroller Unit (MCU): the brain of the board. [chip: ATmega328P]. It stores the program and executes the instructions.

  USB port: [Type-B] Provides power to the board. Allows you to upload your code (sketches) to the MCU and enables serial communication for debugging and sending data.

  Power Input: power the board from an external source (like a wall adapter), when it's not connected to the computer via USB. The Uno can take an input voltage between 6-20V.

- ## How Arduino Runs a program, i.e. upload code then run it

  Writing the code (software, sketch) in Arduino IDE. Based on C/C++

  Auto-compiling: the IDE converts human-readable sketches into machine code.

  Uploading: the compiled code is sent from the computer to the board (hardware) through the USB port. Part of MCU handles receiving and storing this new code into MCU's flash memory.

  Running: the MCU runs the program once uploaded. If the board is disconnected from the computer and reconnected to power, the program stored in its memory will run automatically.

- ## What pins are and how they're labeled

  Pins: they are the physical connectors on the board that allow the Arduino to connect to and interact with external circuits.

  The pins are labeled to indicate their function: A000066-full-pinout.pdf

  - **Digital Pins** (D0 - D13): used for digital input or output. Pins D3, D5, D6, D9, D10, and D11 are also **PWM** pins, which are often used for timers - The PWM functionality is implemented using the microcontroller's internal timers.
  - **Analog Pins** (A0/D14/PC0 to A5/D19/PC5): used primarily for analog input. They can also be used as digital input/output pins (D14 - D19, respectively).
  - **Power and Ground Pins** (e.g. GND, IOREF, +3V3 OUT, +5V OUT, VIN IN)
  - **Communication Pins**: SDA - serial data; SCL - serial clock; dedicated pins for I2C communication.

- ## Digital inputs and outputs (on/off signals)

  Digital input: used to read a signal that can only be in one of two states - HIGH (logic 1, typically 5V), and LOW (logic 0, typically 0V/Ground). Used for simple on/off devices, i.e. buttons or switches.

Digital output: used to send a signal that is either HIGH or LOW, to control simple devices like an LED or a relay switch.

- ## Analog inputs (reading sensor values)

  Analog signals: have a continuous range of values, e.g., the voltage from a temperature sensor that gradually changes.

  Function process: the Arduino's analog input pins have a built-in Analog-to-Digital Converter (ADC). This converts the incoming analog voltage (0V - 5V) into a digital number (0 - 1023) that the MCU can use.
  On standard 10-bit Arduinos, analogRead()()` converts voltage into a 0-1023 integer range, representing 0V to the supply voltage.

- ## PWM outputs (fake analog signals)

  Pulse Width Modulation: Digital pins marked with a tilde (D3, D5, D6, D9, D10, D11) can simulate an analog output.

  It is a technique used by microcontrollers to simulate analog behavior by switching a digital signal ON and OFF at high speeds. By varying the duty cycle (the percentage of time the signal is 'high' vs 'low'), the average power supplied to a device is adjusted to act like a variable voltage.

  Duty cycle: the ratio of ON time to the total period. 100% duty cycle (always ON) = Full brightness on an LED; 50% duty cycle (ON half the time) = half brightness on an LED.

  Fake analog mechanism: PWM does not output true variable voltage. Instead, it rapidly switches between 0V and 5V.

- ## Reading sensors (temperature, buttons)

  Used to connect external components to digital or analog pins and then interpret their states. Buttons act as binary inputs, while many temperature sensors provide analog voltage changes or digital serial data.

  Buttons: read by using digital input pins, requiring a pull-up resistor to prevent a floating signal when the button is open.

  Digital / Analog Temperature Sensors: A digital thermometer measures the temperature internally and outputs the reading as a digital word, requiring only one data pin for communication.

  Related code:

  - `pinMode(pin, INPUT)`: Configures a pin to read input.
  - `digitalRead(pin)`: Returns HIGH (5V) or LOW (0V).
  - `analogRead(pin)`: Returns a value between 0 and 1023 (0V-5V).

- `INPUT_PULLUP`: Activates the internal pull-up resistor to simplify button wiring.

- ## Controlling outputs (LEDs, motors, relays)

LEDs: controlled using digital output pins (HIGH/LOW) or PWM pins to control brightness - turning ON/OFF, or dimming

- Command example:
  `pinMode(13, OUTPUT); digitalWrite(13, HIGH);`
  `analogWrite(3, 128);` (50% brightness)

Motors: Small motors are often controlled using PWM pins to adjust speed. However, motors usually require more current that the Arduino can safely supply, so a separate component (e.g. Motor driver shield) is typically used as an intermediary to handle the power, while the Arduino only sends the control signal.

Relays: An electronic switch controlled by a low-power signal from the Arduino, to turn a high-power circuit (e.g. an AC device) ON/OFF. They are controlled by a simple digital output (HIGH/LOW).

- Command example:
  `digitalWrite(relayPin, LOW)` usually activates the module (on many modules, low is active).

- ## Serial communication (USB serial monitor)

Purpose: To exchange data between the Arduino board and the computer.

Often used for:
Debugging - the Arduino can print text or sensor readings to the Serial Monitor window in the IDE.
Control - can send commands or data from computer back to Arduino (e.g. set a target temperature).

Hardware (How it works): this communication uses RX (Receives data from the computer) and TX (Transmit data to the computer) pins on the board (D0 and D1). The converter chip translates the serial data into the USB format. Then the Arduino IDE's serial monitor opens, reads and displays the coming data, allowing you to type and send data back to the board.

- ## I2C & SPI communication types (including busses)

The two common types of serial bus protocols that allow the Arduino to communicate with multiple sensors or modules simultaneously.

I2C: two-wire interface
It uses two lines: SDA (data) - to send and receive data, and SCL (clock) - to synchronize the communication.

Often used for short-distance, low-speed communication with devices like real-time clocks or specific sensors.

SPI: serial peripheral interface
It uses four lines (pins): SCK, MOSI, MISO, SS
Used for faster, full-duplex (simultaneous send and receive) communication.

3. Arduino Modules - typical inputs, expected outputs and setup (specific to the PODs)

- Motor controller (function only)

- Clock module/RTC (function only)
- SD module (function only)
- Prototyping board (in the context of the DS18B20 3-wire temperature sensors)
- Data Logging Shield with prototyping area
- Motor Driver Shield
- Sensor expansion shield (see product description)
- LED display Z6524A.pdf